Module 1:
Introduction to AI and Intelligent Agents

1. Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

Code:

```
class TableDrivenVacuumAgent:
    def __init__(self):
        # The percept history of the agent
        self.percept_history = []

        # Look-up table: Maps a sequence of percepts to a specific action
        # Percept format: (Location, Status)
        self.table = {
            # Time step 1: Single percept sequences
            ((('A', 'Clean'),),): 'Right',
            ((('A', 'Dirty'),),): 'Suck',
            ((('B', 'Clean'),),): 'Left',
            ((('B', 'Dirty'),),): 'Suck',

            # Time step 2: Two percept sequences (examples of history)
            ((('A', 'Clean'),), (('B', 'Dirty'),)): 'Suck',
            ((('A', 'Clean'),), (('B', 'Clean'),)): 'Right',  # Patrolling
            ((('A', 'Dirty'),), (('A', 'Clean'),)): 'Right',  # Sucked, now move
            ((('B', 'Dirty'),), (('B', 'Clean'),)): 'Left',   # Sucked, now move

            # Time step 3: Three percept sequences
            ((('A', 'Dirty'),), (('A', 'Clean'),), (('B', 'Dirty'),)): 'Suck',
            ((('A', 'Dirty'),), (('A', 'Clean'),), (('B', 'Clean'),)): 'Left',
        }

    def act(self, current_percept):
        """Appends current percept to history and looks up the next action."""
        self.percept_history.append(current_percept)

        # Convert history list to a tuple of tuples so it can be used as a dictionary key
        history_key = tuple(self.percept_history)

        # Look up action in table; default to 'Right' if sequence isn't explicitly mapped
        action = self.table.get(history_key, 'Right')
        return action
```

```
# --- Simulation / Environment Loop ---
if __name__ == "__main__":
    agent = TableDrivenVacuumAgent()

    # Simulating a sequence of steps in the environment
    # Step 1: Agent starts in Room A, which is Dirty
    percept_1 = ('A', 'Dirty')
    action_1 = agent.act(percept_1)
    print(f"Percept: {percept_1} -> Action: {action_1}")

    # Step 2: Agent sucked the dirt, so Room A is now Clean
    percept_2 = ('A', 'Clean')
    action_2 = agent.act(percept_2)
    print(f"Percept: {percept_2} -> Action: {action_2}")

    # Step 3: Agent moved Right and finds Room B is Dirty
    percept_3 = ('B', 'Dirty')
    action_3 = agent.act(percept_3)
    print(f"Percept: {percept_3} -> Action: {action_3}")
```

Output:

```
Percept: ('A', 'Dirty') -> Action: Right
Percept: ('A', 'Clean') -> Action: Right
Percept: ('B', 'Dirty') -> Action: Right
```

Problem Solving by Searching (Uninformed Search)

2. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

Code:

```
from collections import deque

class EightPuzzleBFS:
    def __init__(self, start_state, goal_state):
        self.start_state = tuple(start_state)
        self.goal_state = tuple(goal_state)

        # Valid moves mapped to index shifts in a 1D array representing a 3x3 grid
        # Format: (index_delta, direction_name)
        self.moves = {
            'Up': (-3, 'Up'),
            'Down': (3, 'Down'),
```

```
        'Left': (-1, 'Left'),
        'Right': (1, 'Right')
    }

def get_neighbors(self, state):
    """Generates all valid next states from the current state."""
    neighbors = []
    zero_idx = state.index(0)  # Find the empty tile

    # Determine 2D row and column from 1D index
    row, col = zero_idx // 3, zero_idx % 3

    for move_name, (delta, direction) in self.moves.items():
        new_idx = zero_idx + delta

        # Boundary checks to ensure the move stays on the 3x3 board
        if direction == 'Up' and row == 0: continue
        if direction == 'Down' and row == 2: continue
        if direction == 'Left' and col == 0: continue
        if direction == 'Right' and col == 2: continue

        # Swap the 0 with the target tile to create the new state
        state_list = list(state)
        state_list[zero_idx], state_list[new_idx] = state_list[new_idx], state_list[zero_idx]
        neighbors.append((tuple(state_list), move_name))

    return neighbors

def solve(self):
    """Finds the shortest path from start_state to goal_state using BFS."""
    if self.start_state == self.goal_state:
        return []

    # Queue stores: (current_state, path_of_moves_taken)
    queue = deque([(self.start_state, [])])

    # Set to track visited states and prevent infinite loops
    visited = {self.start_state}

    while queue:
        current_state, path = queue.popleft()

        # Explore all legal moves from the current configuration
        for next_state, action in self.get_neighbors(current_state):
```

```
            if next_state == self.goal_state:
                return path + [action]

            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [action]))

        return None  # Return None if no solution exists

def print_board(state):
    """Helper function to print the 3x3 board cleanly."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)


# --- Execution Example ---
if __name__ == "__main__":
    # 0 represents the empty space
    # Simple configuration solvable in a few steps
    initial_configuration = (1, 2, 3,
                             4, 0, 5,
                             7, 8, 6)

    goal_configuration    = (1, 2, 3,
                             4, 5, 6,
                             7, 8, 0)

    print("Initial State:")
    print_board(initial_configuration)

    solver = EightPuzzleBFS(initial_configuration, goal_configuration)
    solution = solver.solve()

    if solution is not None:
        print(f"Success! Shortest path found in {len(solution)} moves.")
        print(f"Sequence of moves: {solution}")
    else:
        print("This configuration is unsolvable.")
Output:
```

```
Initial State:
 1 2 3
 4 0 5
 7 8 6
-----------
Success! Shortest path found in 2 moves.
Sequence of moves: ['Right', 'Down']
```

3. Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

Code:

```
# Configure capacities and target dynamically
J1_CAP = 4  # Capacity of Jug 1
J2_CAP = 3  # Capacity of Jug 2
TARGET = 2  # Desired target volume in either jug


def get_next_states(state):
    """Generates all legal next steps from the current configuration."""
    j1, j2 = state
    moves = []

    # 1. Fill a jug completely
    moves.append((J1_CAP, j2))  # Fill Jug 1
    moves.append((j1, J2_CAP))  # Fill Jug 2

    # 2. Empty a jug completely
    moves.append((0, j2))  # Empty Jug 1
    moves.append((j1, 0))  # Empty Jug 2

    # 3. Pour from Jug 1 to Jug 2
    pour_to_j2 = min(j1, J2_CAP - j2)
    moves.append((j1 - pour_to_j2, j2 + pour_to_j2))

    # 4. Pour from Jug 2 to Jug 1
    pour_to_j1 = min(j2, J1_CAP - j1)
    moves.append((j1 + pour_to_j1, j2 - pour_to_j1))

    # Clean up duplicate or unchanged moves
    return list(set(moves))
```

```
def solve_water_jug_dfs():
    # Stack stores tuples of: (current_state, path_taken)
    # Start with both jugs completely empty: (0, 0)
    stack = [((0, 0), [(0, 0)])]
    visited = {(0, 0)}

    while stack:
        (j1, j2), path = stack.pop()  # Pop the latest element (LIFO behavior)

        # Check if either jug has reached the target quantity
        if j1 == TARGET or j2 == TARGET:
            return path

        for next_state in get_next_states((j1, j2)):
            if next_state not in visited:
                visited.add(next_state)
                # Append to stack to dive deeper down this new branch
                stack.append((next_state, path + [next_state]))

    return None  # No solution possible


# Execute and print path
solution = solve_water_jug_dfs()

if solution:
    print(f"Solution found using DFS ({len(solution)-1} steps):")
    for step, (j1, j2) in enumerate(solution):
        print(f"Step {step}: Jug1 = {j1}L, Jug2 = {j2}L")
else:
    print(f"It is impossible to get exactly {TARGET}L with these jugs.")
```

Output:

```
Solution found using DFS (4 steps):
Step 0: Jug1 = 0L, Jug2 = 0L
Step 1: Jug1 = 0L, Jug2 = 3L
Step 2: Jug1 = 3L, Jug2 = 0L
Step 3: Jug1 = 3L, Jug2 = 3L
Step 4: Jug1 = 4L, Jug2 = 2L
```

Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other weighted graph for applying UCS]

Code:

```
#citydistanceprogram
import heapq


# 1. Define the weighted graph representing Romanian cities and distances
# Graph is represented as an adjacency list: { City: [(Neighbor, Cost), ...] }
# Alternate testing graph
ROMANIA_MAP = {
    "Arad": [("Zerind", 75), ("Sibiu", 140), ("Timisoara", 118)],
    "Zerind": [("Arad", 75), ("Oradea", 71)],
    "Oradea": [("Zerind", 71), ("Sibiu", 151)],
    "Timisoara": [("Arad", 118), ("Lugoj", 111)],
    "Lugoj": [("Timisoara", 111), ("Mehadia", 70)],
    "Mehadia": [("Lugoj", 70), ("Drobeta", 75)],
    "Drobeta": [("Mehadia", 75), ("Craiova", 120)],
    "Craiova": [("Drobeta", 120), ("Rimnicu Vilcea", 146), ("Pitesti", 138)],
    "Sibiu": [("Arad", 140), ("Oradea", 151), ("Fagaras", 99), ("Rimnicu Vilcea", 80)],
    "Rimnicu Vilcea": [("Sibiu", 80), ("Craiova", 146), ("Pitesti", 97)],
    "Fagaras": [("Sibiu", 99), ("Bucharest", 211)],
    "Pitesti": [("Rimnicu Vilcea", 97), ("Craiova", 138), ("Bucharest", 101)],
    "Bucharest": [("Fagaras", 211), ("Pitesti", 101)],
}
# 2. Uniform Cost Search Implementation
def uniform_cost_search(graph, start, goal):
    # Priority Queue stores tuples of: (cumulative_cost, current_node, path_taken)
    priority_queue = [(0, start, [start])]

    # Keep track of visited nodes to avoid processing longer paths to the same city
    visited = set()

    while priority_queue:
        # Pull out the entry with the absolute lowest cumulative cost
        cost, current_node, path = heapq.heappop(priority_queue)

        # The first time we pop the goal node, it is guaranteed to be the least-cost path
        if current_node == goal:
            return path, cost
```

```
        if current_node in visited:
            continue
        visited.add(current_node)

        # Explore neighbors
        for neighbor, edge_cost in graph.get(current_node, []):
            if neighbor not in visited:
                total_cost = cost + edge_cost
                heapq.heappush(priority_queue, (total_cost, neighbor, path + [neighbor]))

    return None, float('inf')

# 3. Run the UCS search
start_city = "Arad"
goal_city = "Bucharest"
optimal_path, total_cost = uniform_cost_search(ROMANIA_MAP, start_city, goal_city)
```

Output:

```
Least-cost path from Arad to Bucharest:
Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Distance/Cost: 418 km
```

Problem Solving by Searching (Informed Search)

1. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

Code:

```
import heapq

# Define the goal state (0 represents the blank space)
GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

# Target positions (row, col) for each tile (0-8) to calculate Manhattan distance
GOAL_POSITIONS = {tile: (i // 3, i % 3) for i, tile in enumerate(GOAL_STATE)}

def get_manhattan_distance(state):
    """Calculates the sum of Manhattan distances of tiles from their goal positions."""
    distance = 0
    for i, tile in enumerate(state):
        if tile != 0:  # Skip the blank tile
            current_row, current_col = i // 3, i % 3
            goal_row, goal_col = GOAL_POSITIONS[tile]
```

```
            distance += abs(current_row - goal_row) + abs(current_col - goal_col)
    return distance

def get_neighbors(state):
    """Generates next valid states by moving the blank tile (0)."""
    neighbors = []
    blank_idx = state.index(0)
    row, col = blank_idx // 3, blank_idx % 3

    # Define possible moves: (row_offset, col_offset)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right

    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank_idx = new_row * 3 + new_col
            # Convert to list to swap tiles, then back to tuple
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx],
new_state[blank_idx]
            neighbors.append(tuple(new_state))

    return neighbors

def greedy_best_first_search(start_state):
    """Solves the 8-puzzle using Greedy Best-First Search."""
    # Priority queue stores tuples of: (heuristic_value, current_state, path_taken)
    # heapq orders by the first element (heuristic_value)
    open_list = []
    heapq.heappush(open_list, (get_manhattan_distance(start_state), start_state, [start_state]))

    visited = set()
    visited.add(start_state)

    while open_list:
        _, current_state, path = heapq.heappop(open_list)

        # Check if goal is reached
        if current_state == GOAL_STATE:
            return path

        # Explore neighbors
        for neighbor in get_neighbors(current_state):
            if neighbor not in visited:
```

```
                visited.add(neighbor)
                h_value = get_manhattan_distance(neighbor)
                heapq.heappush(open_list, (h_value, neighbor, path + [neighbor]))

    return None  # No solution found

def print_state(state):
    """Helper function to print the grid format nicely."""
    for i in range(0, 9, 3):
        print(list(state[i:i+3]))
    print()

# --- Example Usage ---
if __name__ == "__main__":
    # 0 represents the empty space
    # An example initial state that is solvable
    initial_configuration = (1, 2, 3,
                             0, 4, 6,
                             7, 5, 8)

    print("Initial State:")
    print_state(initial_configuration)

    solution_path = greedy_best_first_search(initial_configuration)

    if solution_path:
        print(f"Solution found in {len(solution_path) - 1} moves!\n")
        for step, state in enumerate(solution_path):
            print(f"Step {step}:")
            print_state(state)
    else:
        print("No solution could be found.")
```

Output:

```
Initial State:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Solution found in 3 moves!

Step 0:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Step 1:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 3:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

2. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest path using A* search with the Manhattan Distance heuristic.

Bayes' Rule Application

1. **Scenario:**
   A medical test is used to detect a particular disease.
   The probability that a randomly selected person has the disease is 1%.
   If a person has the disease, the test returns positive with probability 99%.
   If a person does not have the disease, the test still returns positive with probability 5% (false positive rate).
   A person takes the test and receives a positive result.
   Problem Statement:
   Write a Python program to compute the probability that the person actually has the disease given that the test result is positive.

Code:

```
def calculate_disease_probability():
    # Given data
    prob_disease = 0.01          # P(D)
    prob_no_disease = 0.99       # P(D')

    true_positive_rate = 0.99    # P(T+|D)
    false_positive_rate = 0.05   # P(T+|D')
```

```
    # 1. Calculate the total probability of a positive test result P(T+)
    prob_positive_test = (true_positive_rate * prob_disease) + (false_positive_rate *
prob_no_disease)

    # 2. Apply Bayes' Theorem to calculate P(D|T+)
    prob_disease_given_positive = (true_positive_rate * prob_disease) / prob_positive_test

    # Display the results
    print(f"Total probability of testing positive P(T+): {prob_positive_test:.4f}")
    print(f"Probability of actually having the disease given a positive test P(D|T+):
{prob_disease_given_positive:.4f}")
    print(f"In percentage: {prob_disease_given_positive * 100:.2f}%")

if __name__ == "__main__":
    calculate_disease_probability()
```

Output:

```
Total probability of testing positive P(T+): 0.0594
Probability of actually having the disease given a positive test P(D|T+): 0.1667
In percentage: 16.67%
```

Naïve Bayes' Classification

1. Using a small dataset (e.g., weather or spam dataset), implement a Naive Bayes classifier using scikit-learn.

Code:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

# 1. Create a tiny dataset (Text, and Label: 1 for spam, 0 for ham)
corpus = [
    "Get rich quick! Click here now!",  # Spam
    "Hey, are we still meeting for lunch today?",  # Ham
    "Free cash prize! Claim your reward.",  # Spam
    "Can you send me the project report?",  # Ham
    "Urgent: Account suspended. Verification required.",  # Spam
    "Don't forget to buy milk on your way home.",  # Ham
]
labels = [1, 0, 1, 0, 1, 0]

# 2. Convert text into numerical vectors (Bag of Words)
```

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
y = labels

# 3. Train the Naive Bayes model (MultinomialNB is great for text)
model = MultinomialNB()
model.fit(X, y)

# 4. Test the model with brand new emails
test_emails = [
    "Hey buddy, let's grab a coffee later.",
    "CONGRATULATIONS! You won a free vacation! Click now",
]

# Convert test text using the same vectorizer
X_test = vectorizer.transform(test_emails)
predictions = model.predict(X_test)

# 5. Output the results
for email, pred in zip(test_emails, predictions):
    status = "SPAM 🚨" if pred == 1 else "HAM (Safe) ✅"
    print(f"Email: '{email}' --> Predicted as: {status}")
```

Output:

```
Email: 'Hey buddy, let's grab a coffee later.' --> Predicted as: HAM (Safe) ✅
Email: 'CONGRATULATIONS! You won a free vacation! Click now' --> Predicted as: SPAM 🚨
```

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.
2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code:

```
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
from pgmpy.models import DiscreteBayesianNetwork

# 1. Define the network structure
model = DiscreteBayesianNetwork([("Burglary", "Alarm"), ("Earthquake",
"Alarm")])
```

```
# 2. Define Conditional Probability Distributions (CPDs)
# States are ordered as: [False, True]
cpd_b = TabularCPD(
    variable="Burglary", variable_card=2, values=[[0.99], [0.01]]
)
cpd_e = TabularCPD(
    variable="Earthquake", variable_card=2, values=[[0.98], [0.02]]
)

# Column mapping for evidence [Burglary, Earthquake]:
# [False,False], [False,True], [True,False], [True,True]
cpd_a = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.06, 0.71, 0.05],  # P(Alarm = False)
        [0.001, 0.94, 0.29, 0.95],  # P(Alarm = True)
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
)

# 3. Attach CPDs to the model
model.add_cpds(cpd_b, cpd_e, cpd_a)

# 4. Compute Posterior Probability: P(Burglary | Alarm = True)
inference = VariableElimination(model)
posterior = inference.query(variables=["Burglary"], evidence={"Alarm": 1})

print(posterior)
```

Output:

```
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.8659 |
+-------------+-----------------+
| Burglary(1) |          0.1341 |
+-------------+-----------------+
```

Machine Learning

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.
b. Perform basic preprocessing (handling missing values or scaling).
c. Split dataset into training and testing sets.
d. Train a simple classifier.
e. Display training and testing accuracy.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

# ======================================================================
# a. Load dataset using Pandas
# ======================================================================
# We load the Iris dataset from sklearn and convert it into a Pandas DataFrame
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df["target"] = iris.target

print("--- Step A: Dataset Loaded ---")
print(df.head(), "\n")

# Separate features (X) and target label (y)
X = df.drop(columns=["target"])
y = df["target"]

# ======================================================================
# b. Perform basic preprocessing
# ======================================================================
# 1. Handle missing values (if any) by filling them with the column mean
X = X.fillna(X.mean())

# 2. Scale features so they have a mean of 0 and variance of 1
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
print("--- Step B: Preprocessing Complete ---")
print(f"Sample scaled feature row: {X_scaled[0]}\n")

# ========================================================================
# c. Split dataset into training and testing sets
# ========================================================================
# 80% of data goes to training, 20% goes to testing
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print("--- Step C: Dataset Split ---")
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples\n")

# ========================================================================
# d. Train a simple classifier
# ========================================================================
# Using a Support Vector Machine (SVM) classifier
classifier = SVC(kernel="linear", random_state=42)
classifier.fit(X_train, y_train)

print("--- Step D: Model Training Complete ---\n")

# ========================================================================
# e. Display training and testing accuracy
# ========================================================================
# Make predictions on both sets
y_train_pred = classifier.predict(X_train)
y_test_pred = classifier.predict(X_test)

# Calculate accuracy scores
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("--- Step E: Final Results ---")
print(f"Training Accuracy: {train_accuracy * 100:.2f}%")
print(f"Testing Accuracy:  {test_accuracy * 100:.2f}%")
```

Output:

```
--- Step A: Dataset Loaded ---
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1               3.5  ...               0.2       0
1                4.9               3.0  ...               0.2       0
2                4.7               3.2  ...               0.2       0
3                4.6               3.1  ...               0.2       0
4                5.0               3.6  ...               0.2       0

[5 rows x 5 columns]

--- Step B: Preprocessing Complete ---
Sample scaled feature row: [-0.90068117  1.01900435 -1.34022653 -1.3154443 ]

--- Step C: Dataset Split ---
Training set size: 120 samples
Testing set size: 30 samples

--- Step D: Model Training Complete ---

--- Step E: Final Results ---
Training Accuracy: 98.33%
Testing Accuracy:  96.67%
```

2. Using a suitable regression dataset, implement Linear Regression using scikit-learn.

Code:

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# 1. Load the Diabetes regression dataset
diabetes = load_diabetes()
X = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
y = diabetes.target  # Continuous target variable (disease progression)

# Let's use a single strong feature like 'bmi' for simple visualization/intuition
X_bmi = X[["bmi"]]

# 2. Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X_bmi, y, test_size=0.2, random_state=42
)

# 3. Initialize and train the Linear Regression model
```

```
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Make predictions on the test set
y_pred = model.predict(X_test)

# 5. Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("--- Linear Regression Results ---")
print(f"Model Coefficient (Slope): {model.coef_[0]:.2f}")
print(f"Model Intercept:           {model.intercept_:.2f}")
print(f"Mean Squared Error (MSE):  {mse:.2f}")
print(f"R-squared Score (R²):      {r2:.2f}")
```

Output:

```
--- Linear Regression Results ---
Model Coefficient (Slope): 998.58
Model Intercept:           152.00
Mean Squared Error (MSE):  4061.83
R-squared Score (R²):      0.23
```

3. Using the MNIST dataset (digit vs not digit), implement a binary classifier.

Code:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split

# 1. Load the digits dataset (a smaller, built-in version of MNIST)
digits = load_digits()
X = digits.data  # Flattened pixel arrays (64 features per image)
y = digits.target

# 2. Transform into a Binary Classification task: Is it a '5' or not?
# y_binary will be True (1) if the digit is 5, and False (0) for any other digit
y_binary = y == 5

# 3. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y_binary, test_size=0.2, random_state=42
```

```
)

# 4. Train a Logistic Regression binary classifier
model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

# 5. Evaluate the model
y_pred = model.predict(X_test)

print("--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))
print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred, target_names=["Not a 5", "Is a 5"]))
```

Output:

```
--- Confusion Matrix ---
[[310   3]
 [  2  45]]

--- Classification Report ---
              precision    recall  f1-score   support

     Not a 5       0.99      0.99      0.99       313
      Is a 5       0.94      0.96      0.95        47

    accuracy                           0.99       360
   macro avg       0.97      0.97      0.97       360
weighted avg       0.99      0.99      0.99       360
```

4. Implement k-fold cross-validation on a classification model.

Code:

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import KFold, cross_val_score

# 1. Load a sample dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Initialize the classification model
model = RandomForestClassifier(random_state=42)

# 3. Set up the K-Fold strategy (e.g., 5 folds)
```

```
# shuffle=True ensures the data is mixed before splitting
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# 4. Perform K-Fold Cross-Validation
# This automatically handles the splitting, training, and evaluating
scores = cross_val_score(model, X, y, cv=kf, scoring="accuracy")

# 5. Display the results
print("--- 5-Fold Cross-Validation Results ---")
for fold_idx, score in enumerate(scores, 1):
    print(f"Fold {fold_idx} Accuracy: {score * 100:.2f}%")

print("-" * 38)
print(f"Mean Accuracy:    {scores.mean() * 100:.2f}%")
print(f"Std Deviation:    {scores.std() * 100:.2f}%")
```

Output:

```
--- 5-Fold Cross-Validation Results ---
Fold 1 Accuracy: 100.00%
Fold 2 Accuracy: 96.67%
Fold 3 Accuracy: 93.33%
Fold 4 Accuracy: 93.33%
Fold 5 Accuracy: 96.67%
--------------------------------------
Mean Accuracy:    96.00%
Std Deviation:    2.49%
```

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

Code:

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

# 1. Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 3. Initialize and train the Decision Tree Classifier
# We set max_depth to keep the tree simple and avoid overfitting
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

# 4. Evaluate the model
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Testing Accuracy: {accuracy * 100:.2f}%\n")

# 5. Display the decision rules in text format
print("--- Learned Decision Tree Rules ---")
tree_rules = export_text(model, feature_names=list(iris.feature_names))
print(tree_rules)
```

Output:

```
Testing Accuracy: 100.00%

--- Learned Decision Tree Rules ---
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.65
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.75
|   |   |   |--- class: 2
```

6. Implement Decision Tree regression on a regression dataset.

Code:

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor, export_text

# 1. Load the regression dataset
diabetes = load_diabetes()
```

```
X = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
y = diabetes.target

# 2. Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Initialize and train the Decision Tree Regressor
# max_depth keeps the tree compact and prevents severe overfitting
regressor = DecisionTreeRegressor(max_depth=3, random_state=42)
regressor.fit(X_train, y_train)

# 4. Make predictions and evaluate
y_pred = regressor.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("--- Evaluation Metrics ---")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared Score (R²):     {r2:.2f}\n")

# 5. Display a sample of the tree rules
print("--- Learned Regression Rules (First few levels) ---")
tree_rules = export_text(regressor, feature_names=list(diabetes.feature_names))
print(tree_rules)
```

Output:

```
--- Evaluation Metrics ---
Mean Squared Error (MSE): 3552.70
R-squared Score (R²):     0.33

--- Learned Regression Rules (First few levels) ---
|--- bmi <= 0.01
|   |--- s5 <= 0.01
|   |   |--- s5 <= -0.04
|   |   |   |--- value: [80.88]
|   |   |--- s5 >  -0.04
|   |   |   |--- value: [109.92]
|   |--- s5 >  0.01
|   |   |--- s4 <= 0.09
|   |   |   |--- value: [159.57]
|   |   |--- s4 >  0.09
|   |   |   |--- value: [256.33]
|--- bmi >  0.01
|   |--- bmi <= 0.07
|   |   |--- s6 <= 0.03
|   |   |   |--- value: [175.80]
|   |   |--- s6 >  0.03
|   |   |   |--- value: [230.52]
|   |--- bmi >  0.07
|   |   |--- s2 <= 0.02
|   |   |   |--- value: [291.22]
|   |   |--- s2 >  0.02
|   |   |   |--- value: [225.75]
```

7. Implement k-Nearest Neighbors (kNN) classifier.

Code:

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

# 1. Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 3. Scale the features
# Crucial for k-NN because it relies on calculating geometric distances
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Initialize and train the k-NN Classifier (using 3 neighbors)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)

# 5. Make predictions and evaluate accuracy
y_pred = knn.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)

print(f"k-NN Classifier Accuracy (k=3): {accuracy * 100:.2f}%")
```

Output:

```
k-NN Classifier Accuracy (k=3): 100.00%
```

8. Implement multiclass classification using Logistic Regression or Decision Tree.

Code:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load the Iris dataset (3 target classes: 0, 1, or 2)
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Scale features (Highly recommended for Logistic Regression convergence)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# 4. Train a Multiclass Logistic Regression model
# multi_class='ovr' trains a separate binary classifier for each class
model = LogisticRegression(multi_class="ovr", random_state=42)
model.fit(X_train_scaled, y_train)

# 5. Evaluate the model
y_pred = model.predict(X_test_scaled)

print("--- Multiclass Classification Report ---")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Output:

```
--- Multiclass Classification Report ---
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.89      0.94         9
   virginica       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```